

UNIT-5

1. introduction

In C++, string representation primarily involves the use of the `std::string` class from the Standard Library, which provides a flexible way to handle strings.

Key Features of `std::string`:

- i. **Dynamic Size:** Unlike C-style strings (character arrays), `std::string` can grow and shrink in size dynamically.
- ii. **Memory Management:** It automatically manages memory, reducing the risk of memory leaks and buffer overflows.
- iii. **Member Functions:** It includes various functions for string manipulation, such as:
 - `length()`: Returns the number of characters.
 - `substr()`: Extracts a substring.
 - `find()`: Searches for a substring.
 - `append()`: Adds characters to the end.
 - `insert()`: Inserts characters at a specified position.

Creating and Using Strings:

- i. **Initialization:** You can initialize a string using:

```
std::string str1 = "Hello, World!";  
std::string str2(10, 'A'); // Creates a string with 10 'A's
```

- ii. **Concatenation:** Use the `+` operator or `append()` method:

```
std::string str3 = str1 + " " + str2;
```

C-style Strings:

- i. C++ also supports C-style strings, which are arrays of characters terminated by a null character (`'\0'`).
- ii. They require manual memory management and are less safe compared to `std::string`.

Conversion Between Types:

- i. You can convert between `std::string` and C-style strings using:
- ii. `c_str()`: Converts `std::string` to a C-style string.

iii. Constructor: Converts a C-style string to std::string.

Example Code:

```
#include <iostream>

#include <string>

int main() {

    std::string greeting = "Hello"; greeting += ", World!";

    std::cout << greeting << std::endl; // Outputs: Hello, World!
    return 0;

}
```

What is std::string?

The std::string is a class in C++ since C++98. This class is the standard representation for a text string. It includes some typical string operations like find, replace, concatenate, compare etc. It is present in <string> header file.

Declaration and Initialization of String

```
string company = "ABESEC Ghaziabad";
```

Commonly Used String Functions in C++

The std::string class contains functions to provide some common string operations. The below table contains some of the most commonly used functions in C++:

S. No.	Category	Functions and Operators	Functionality
1.	String Length	length() or size()	It will return the length of the string.
2.	Accessing Characters	Indexing (using array[index])	To access individual characters using array indexing.
S. No.	Category	Functions and Operators	Functionality

3.	Appending and Concatenating Strings	+ Operator <u>append()</u>	+ operator is used to concatenate two strings. The append() function adds one string to the end of another.
4.	String Comparison	== Operator <u>compare()</u>	You can compare strings using the == operator. The compare() function returns an integer value indicating the comparison result.
5.	Substrings	<u>substr()</u>	Use the substr() function to extract a substring from a string.
6.	Searching	<u>find()</u>	The find() function returns the position of the first occurrence of a substring.
7.	Modifying Strings	<u>replace()</u> <u>insert()</u>	Use the replace() function to modify a part of the string. The insert() function adds a substring at a specified position.

S. No.	Category	Functions and Operators	Functionality
		<u>erase()</u>	Use the erase() function to remove a part of the string.
8.	Conversion	<u>c_str()</u>	To obtain a C-style string from a std::string, you can use the c_str() function.

1. String Length - length() or size()

We can find the length of the string (number of characters) using either **length()** or **size()** function of the std::string class.

Syntax:

```
string_object.size() or string_object.length()
```

Parameters

This function does not take any parameter.

Return Value

This function returns the number of characters in the string object.

Example

```
std::string text = "ABESEC Ghaziabad";
int length = text.length();
// or
int length = text.size();
```

It will return the length of the string **text** which is 16.

2. Accessing Characters - at()

Generally, we can access the character of a string using the **[] array subscript operator** and indexing. But std::string also has a function named **at()** which can be used to access the characters of the string.

Syntax

```
string_object.at(index);
```

Parameters

index: It represents the position of the character in the string.

Return Value

This function returns the character present at the **index**.

Example

```
std::string str = "ABESEC Ghaziabad";  
std::cout << str.at(3);
```

The std::cout will print E on the console as it is the character present at index 3.

3. Concatenating Strings - append() or + Operator

We can concatenate string in C++ using two methods:

1. + Operator

The + operator is overloaded in the std::string class to perform string concatenation.

Syntax

```
string_object1 + string_object2
```

Example

```
std::string firstName = "ABESEC";  
std::string lastName = "Ghaziabad";  
std::string fullName = firstName + " " + lastName;
```

+ operator is used to concatenate two strings. The string fullName will be "ABESEC Ghaziabad".

2. append()

The append() function is another member function to concatenate two strings.

Syntax

```
string_object1.append(string2)
```

Parameters

string2: This function takes the string to be appended as a parameter. It can be both C or C++ Style string.

Return Value

Reference to the final string.

```
std::string base = "Hey! Geeks";  
  
base.append(" Welcome to ABESEC Ghaziabad"); // Append a  
string
```

The append() function adds one string to the end of another.

4. String Comparison - compare() or == Operator

Just like the concatenation, we can do the string comparison using two methods:

1. == Operator

The equality operator can be used to compare the two strings as it is overloaded for this

operation in the `std::string` class.

Syntax

```
string_object1 == string_object2
```

This will return **true** if both the strings are equal, otherwise returns **false**.

Example

```
std::string str1 = "apple";
std::string str2 = "banana";
if (str1 == str2) {
    std::cout << "Strings are equal";
}
else {
    std::cout << "Strings are not equal";
}
```

Here, "Strings are not equal" will be printed as the `==` operator will return **false**.

2. compare()

The `compare()` function is a member function of `std::string` class which can be used to compare two strings.

Syntax

```
str1.compare(str2);
```

Parameters

str2: It is the string to be compared. It can be both C or C++ style string.

Return Value

- (i) If the strings are equal, return **zero**.
- (ii) If `str1` is greater than `str2`, return value **>0**
- (iii) If `str2` is greater than `str1`, return value **<0**

Example

```
string str1 = "ABESEC";
string str2 = "ABESECFor";
int result = str1.compare(str2);
```

The result will contain a value less than zero as `str2` is greater than `str1`.

We can also compare the substring of `str2` using the `compare()` function:

```
str1.compare(position, length, str2);
```

where

- (i) **position:** position of the first character substring.
- (ii) **length:** length of the substring.
- (iii) **str2:** String object to be compared.

5. Searching - find()

We can use the **find()** function of the `std::string` class to check whether a given character or a substring is present in the string or a part of string.

Syntax of find()

```
str1.find(var);
```

Parameters

- **var:** It can be a C style string, C++ style string, or a character that is to be searched in the string.

Return Value

- It returns the pointer to the first occurrence of the character or a substring in the string.

Example

```
std::string text = "C++ Programming";
```

```
// Find the position of a substring
```

```
int position = text.find("Programming");
```

The position variable will contain 4 which is the start of the first occurrence of the string "Programming" in string text.

6. Generate Substring - substr()

We can use the **substr() function** to generate a part of the string as a new string object. It is a member function of the `std::string` class.

Syntax of substr() in C

```
str1.substr(start, length);
```

Parameters

- (i) **start:** Starting position of the substring to be generated.
- (ii) **length:** Ending of the substring to be generated.

Return Type

- It returns the newly created string object.

Example

```
std::string text = "Hello, World!";  
std::string sub = text.substr(7, 5);
```

In the above example, the **sub** string will contain the "World".

Modifying Strings

The following function allows us to modify the current string.

1. insert()

The insert() function not only allows us to add a string but also allows us to add it at the specified position. It is also a member function of the std::string class.

Syntax

```
str1.insert(index, str2);
```

Parameters

- (i) **str2**: string to be inserted.
- (ii) **index**: position of where to insert the new string

Return Type

- Reference to the str1.

Example

```
std::string text = "I have a cat.";
```

2. replace()

The replace() function replaces the part of the string with the given other string. Unlike insert, the characters in the part where the new string is to be inserted are removed.

Syntax

```
str1.replace(index, size, str2);
```

Parameters

- (i) **index**: Index of where to start replacing the new string.
- (ii) **size**: length of the part of the string that is to be replaced.
- (iii) **str2**: new string that is to be inserted.

Return Type

Reference to the str1.

Example

```
std::string text = "I like dogs.";
```

```
// Replace "dogs" with "cats"
```

```
text.replace(7, 4, "cats");
```

3. erase()

The erase() function is a member function of std::string class that is used to remove a

character or a part of the string.

Syntax

```
str1.erase(start, end);
```

Parameters

- (i) **start:** Starting position.
- (ii) **end:** Ending position.

Return Type

- Reference to the str1.

Example

```
std::string text = "This is an example.";
text.erase(5, 3); // Erase "is "
```

Convert std::string to C String - c_str()

The `c_str()` function is a member function that is used to convert the C++ style string i.e. `std::string` objects to C style string i.e. array of characters.

Syntax

```
str1.c_str()
```

Parameters

This function does not take any parameter.

Return Value

Pointer to the equivalent array of characters.

Example

```
std::string str = "C++";
const char* cstr = str.c_str()
```

Example of String Functions in C++

The below code demonstrate the use of the above specified string functions:

```
// C++ Code to demonstrate various functions available in
// String class
```

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    // Creating and initializing strings string greeting = "Hello,
    World!"; cout << greeting << endl;
    string name;

    // Input from the user
```

```

cout << "Enter your name: ";
cin >> name;
cout << name << endl;

// String length
int length = greeting.length();
cout << length << endl;

// Accessing characters
char firstChar = greeting[0];
char secondChar = greeting.at(1);
cout << firstChar << " " << secondChar << endl;

// Appending and concatenating strings
string firstName = "ABESEC";
string lastName = "Ghaziabad";
string fullName = firstName + " " + lastName;
cout << fullName << endl;
string base = "Hello";
cout << base << endl;
base.append(" World!");
cout << base << endl;

// String comparison
string str1 = "apple";
string str2 = "banana";
if (str1 == str2) {
    cout << "Strings are equal" << endl;
}
else {
    cout << "Strings are not equal" << endl;
}

int result = str1.compare(str2);
if (result == 0) {
    cout << "Strings are equal" << endl;
}
else if (result < 0) {
    cout << "str1 comes before str2" << endl;
}
else {
    cout << "str1 comes after str2" << endl;
}

// Substrings
string text = "Hello, World!";
cout << text << endl;
string sub = text.substr(7, 5);
cout << sub << endl;

```

```

// Searching
string searchIn = "C++ Programming";
size_t position = searchIn.find("Programming");
if (position != string::npos) {
    cout << "Found at position " << position << endl;
}
else {
    cout << "Not found" << endl;
}

// Modifying strings
string modify = "I like dogs.";
modify.replace(7, 4, "cats");
cout << modify << endl;
modify.insert(6, " black");
cout << modify << endl;
modify.erase(6, 6);
cout << modify << endl;
// Conversion string
str = "C++";
const char* cstr = str.c_str();
cout << cstr << endl;
return 0;
}

```

Object Oriented Programming in C++

OOP in C++ was introduced to solve this problem by organizing code into classes and objects, making programs easier to understand, reuse, and maintain.

Key Features of OOP in C++:

- (i) Structures code into logical units (classes and objects)
- (ii) Keeps related data and methods together (encapsulation)
- (iii) Makes code modular, reusable and scalable
- (iv) Prevents unauthorized access to data
- (v) Follows the DRY (Don't Repeat Yourself) principle

1. Class

A class is a user-defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type. Using classes, you can create multiple objects with the same behavior instead of writing their code multiple times. In general, class declarations in C++ can include these components.

- (i) **Access Specifiers:** A class can have members defined as public, private, or protected to control accessibility.
- (ii) **Class Name:** The class name should follow naming conventions, usually starting with a capital letter.
- (iii) **Body:** The class body is enclosed with braces {} and defines data members and member functions.

2. Object

An Object is a basic unit of Object-Oriented Programming that represents real-life entities. A typical C++ program creates many objects, which interact with each other by invoking methods. The objects are what perform your code, they are the part of your code visible to the user. An object mainly consists of:

- (i) **State:** It is represented by the data members (attributes) of an object. It also reflects the properties of an object.
- (ii) **Member Function:** A member function is a collection of statements that perform some specific task and may return the result to the caller.
- (iii) **Behavior:** It is represented by the member functions of an object. It also reflects the response of an object to other objects.
- (iv) **Identity:** It is a unique name or reference given to an object that enables it to interact with other objects.

Example

```
#include <iostream>
#include <string>
using namespace std;

class Employee {

    // Instance variables private:
    string name;
    float salary;

public:
    // Constructor
    Employee(string name, float salary)
    { this->name = name;
      this->salary = salary;
    }

    // getters method
    string getName() {
        return name; }

    float getSalary() {
        return salary; }
```

```

// setters method
void setName(string name) {
    this->name = name;
}
void setSalary(float salary) {
    this->salary = salary;
}
// Instance method
void displayDetails() {
    cout << "Employee: " << name << endl; cout << "Salary: " <<
    salary << endl;
}
};

int main() {
    Employee emp("Anant", 10000.0f); emp.displayDetails();
    return 0;
}

```

Output

```

Employee:Anant
Salary: 10000

```

3. Abstraction

Abstraction in C++ is the process of hiding the implementation details and only showing the essential details or features to the user. It allows to focus on what an object does rather than how it does it.

In C++ abstraction is achieved using abstract classes (classes that have at least one pure virtual function).

Example

```

#include <iostream>
using namespace std;

// Abstract class Vehicle
class Vehicle {
public:

// Abstract methods
virtual void accelerate() = 0;

// pure virtual function virtual void brake() = 0;

void startEngine() {
    cout << "Engine started!" << endl;
}
};

```

```

class Car : public Vehicle { public:

// Implement abstract methods void accelerate() override {
cout << "Car: Pressing gas pedal..." << endl;
}

void brake() override {
cout << "Car: Applying brakes..." << endl;

}
};
int main() {

// Create object using pointer to abstract class
Vehicle* myCar = new Car();
myCar->startEngine();
myCar->accelerate();
myCar->brake();
delete myCar;
return 0;
}

```

Output

```

Engine started!
Car: Pressing gas pedal...
Car: Applying brakes...

```

4. Encapsulation

Encapsulation is defined as the process of wrapping data and the methods into a single unit, typically a class. It is like a protective shield that prevents the data from being accessed by the code outside the shield.

- (i) Technically, in encapsulation the variables or the data in a class is hidden from any other class and can be accessed only through any member function of the class in which they are declared.
- (ii) In encapsulation, the data in a class is hidden from other classes, which is similar to what **data-hiding** does.
- (iii) Encapsulation can be achieved by declaring all the variables in a class as private and writing public methods in the class to set and get the values of the variables.

Example

```

#include <iostream>
#include <string>
using namespace std;
class Employee {

```

```
// Private fields private:
int id;
string name;
public:
// Setter methods
void setId(int id) {
this->id = id;
}
void setName(string name) {
this->name = name;
}

// Getter methods int getId() {
return id;
}

string getName() {
return name;
}
};
```

```

int main() {
    Employee emp;

    // Using setters
    emp.setId(101);
    emp.setName("Anant");

    // Using getters
    cout << "Employee ID: " << emp.getId() << endl;
    cout << "Employee Name: " << emp.getName() << endl;

    return 0;
}

```

Output

```

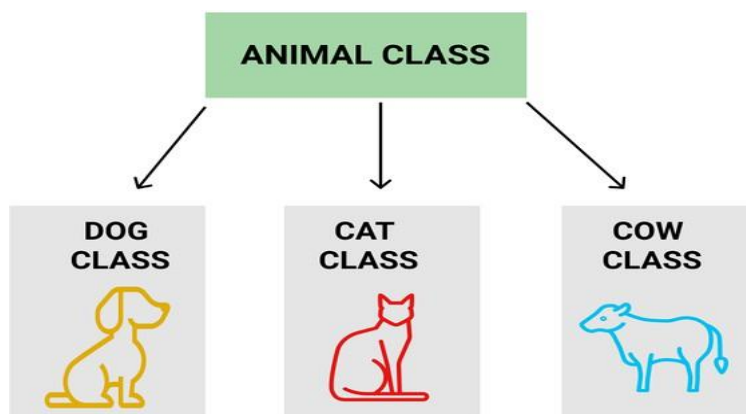
Employee ID: 101
Employee Name: Anant

```

5. Inheritance

Inheritance is an important pillar of OOP (Object Oriented Programming). It is the mechanism in C++ by which one class is allowed to inherit the features (data members and member functions) of another class. We achieve inheritance by using the : symbol followed by an access specifier (public, private, or protected). Inheritance is also known as an "is-a" relationship.

Example: Dog, Cat, Cow and be Derived Class of Animal Base Class.



Let us discuss some frequently used important terminologies:

- (i) **Superclass:** The class whose features are inherited is known as superclass (also known as base or parent class).
- (ii) **Subclass:** The class that inherits the other class is known as subclass (also known as derived or extended or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- (iii) **Reusability:** Inheritance supports the concept of "reusability", i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class.

Example

```
#include <iostream>
using namespace std;
// Superclass (Parent)
class Animal {
public:
void eat() {
cout << "Animal is eating..." << endl;
}
void sleep() {
cout << "Animal is sleeping..." << endl;
}
};

// Subclass (Child) - Inherits from Animal class
Dog : public Animal {
public:
void bark() {
cout << "Dog is barking!" << endl;
}
};

int main() {
Dog myDog;

// Inherited methods (from Animal) myDog.eat();
myDog.sleep();

// Child class method myDog.bark();
return 0;
}
```

Output

```
Animal is eating...
Animal is sleeping...
Dog is barking!
```

6. Polymorphism

The word polymorphism means having many forms, and it comes from the Greek words poly (many) and morph (forms), this means one entity can take many forms. In C++, polymorphism allows the same method or object to behave differently based on the context, specially on the project's actual runtime class.

Types of Polymorphism

Polymorphism in C++ is mainly of 2 types as mentioned below:

- Function Overloading
- Function Overriding

Function Overloading and Function Overriding

1. Function Overloading : Also known as compile-time polymorphism, this occurs when two or ore functions in the same class share the same name but have different parameters lists (different number or types of parameters). The return type of these functions may or may not be same.

2. Function Overriding : Also, known as run-time polymorphism, this occurs when a function in the derived class has the same name, return type, and parameters as a function in the base class. In C++, this is achieve using virtual functions, so that the derived class provides its own implementation of the method.

Below is the implementation of both the concepts:

```
#include <iostream>
using namespace std;
// Parent Class
class Parent { public:

    // Overloaded method
    void func() {
        cout << "Parent.func()" << endl;
    }

    // Overloaded method
    virtual void func(int a) {
        cout << "Parent.func(int): " << a << endl;
    }
};

// Child Class
class Child : public Parent { public:
```

```

// Overrides Parent.func(int)
void func(int a) override {
    cout << "Child.func(int): " << a << endl;
}
};

int main() {
    Parent parent;
    Child child;
    Parent* polymorphicObj = new Child();

    // Method Overloading (compile-time)
    parent.func();
    parent.func(10);

    // Method Overriding (runtime) child.func(20);

    // Polymorphism in action polymorphicObj->func(30); delete
    polymorphicObj; return 0;
}

```

Output

```

Parent.func() Parent.func(int): 10
Child.func(int): 20
Child.func(int): 30

```

Advantage of OOP over Procedure-Oriented Programming Language.

Object oriented programming (OOP) offers several key advantages over procedural programming:

- (i) By using objects and classes, you can create reusable components, leading to less duplication and more efficient development.
- (ii) It provides a clear and logical structure, making the code easier to understand, maintain, and debug.
- (iii) OOP support the DRY (Don't Repeat Yourself) principle. This principle encourages minimizing code repetition, leading to cleaner, more maintainable code.

Disadvantage of OOP

- (i) OOP has concepts like classes, objects, inheritance etc. For beginners, this can be confusing and takes time to learn.
- (ii) If we write a small program, using OOP can feel too heavy. We might have to write more code than needed just to follow the OOP structure.
- (iii) The code is divided into different classes and layers, so in this, finding and fixing bugs can sometimes take more time.